

Centroidal MPC for Inertial Reorientation Using Bio-inspired Tailed Robot

Yichen Wang, Qin Yue

[illegible]

I. INTRODUCTION

Inertial reorientation (IR) refers to the ability of a robot to change its orientation without or with minimal interaction with the ground or other objects. This movement can increase robot agility by stabilizing the robot between ground contacts in gaits with long flight phases, rejecting sudden disturbances, or increasing the speed of the robots turns. A tail-like appendage is shown to be effective in increasing the robot's ability for inertial-reorientation [1].

However, the addition of a tail appendage makes the robot system more complex. Moreover, effective inertial reorientation control frequently necessitates control using the full-body dynamic model of the robot, as opposed to the typical approach of abstracting the robot to a single rigid body. Centroidal MPC can represent a useful compromise of these two methods [2].

II. METHODS

A. Robot Model

We use a bipedal tailed robot model. The robot has eight actuated degrees of freedom, denoted $q_j \in \mathbb{R}^8$: two for the tail, and three for each leg. The tail is modelled as two sequentially actuated joints for pitch and yaw, placed close to the torso, with a long bar and a heavy counterweight connected at the end. Each leg is modeled as three sequential joints for adduction-abduction, hip extension-flexion, and knee-extension-flexion. Additionally, the model includes six

floating base joints, $q_b \in \mathbb{R}^6$, to simulate free movement in three dimensions: three linear joints in the x, y, and z direction, and three rotating joints in the x, y, and z axis.

We used the software package `Spatial V2` [3] to model and visualize this model, shown in figure 2. The parent link, joint properties, mass, and inertia of each link are denoted in Table I, where $p(j)$ denotes the parent link of link i , Position denotes the translation portion of the transform from the predecessor frame to the current link frame, ${}^i\mathbf{X}_p(i)$, CoM Pos refers to the center of mass position in the current link frame, $I \in \mathbb{R}^{3 \times 3}$ refers to the diagonalized link inertia around the CoM. All link frames are constructed with identical orientation to the world frame when $q_j = \mathbf{0}^8$, so frame orientations are omitted. Link 0, or the Torso, is connected to ground via six sequential floating-base joints, using the order $[Px \ Py \ Pz \ Rx \ Ry \ Rz]$. q_b and q_j are constructed with the same order as above. We define $q = [q_b^\top \ q_j^\top]^\top \in \mathbb{R}^{14}$ to be the concatenation of floating-base and actuated joints.

TABLE I
TABLE OF ROBOT PARAMETERS.

<i>i</i>	Name	$p(i)$	Axis	Frame Pos.	M	CoM Pos.	$\text{diag}(I) \times 10^4$
0	Torso	n/a	n/a	[0 0 0]	1	[0 0 .2]	[94 83 27]
1	Tail-Y	0	z	[-0.05, 0, 0]	.05	[-0.025 0 0]	[0 0.104 0.104]
2	Tail-P	1	y	[-0.05, 0, 0]	.5	[-0.27 0 0]	[0 25.5 25.5]
3	AbAd-R	0	x	[0, -.075, 0]	.3	[0 0 0]	[0.625 22.75 22.75]
4	Hip-R	3	y	[0, 0, 0]	.75	[0 0 -0.02]	[18.25 38.8 21.8]
5	Knee-R	4	y	[0, 0, -0.2]	.1	[0 0 -0.1]	[3.33 3.33 0]
6	AbAd-L	0	x	[0, .075, 0]	.3	[0 0 0]	[0.625 22.75 22.75]
7	Hip-L	6	y	[0, 0, 0]	.75	[0 0 -0.02]	[18.25 38.8 21.8]
8	Knee-L	7	y	[0, 0, -0.2]	.1	[0 0 -0.1]	[3.33 3.33 0]

B. Trajectory Optimization Formulation

Reference trajectories for online CM-MPC controller are generated using offline trajectory optimization. We decided to use simple Euler integration for the dynamics, with $dt = 0.02s$, time horizon of $2s$, and $N_{hor} = 100$. We formulated the trajectory optimization problem as the following NLP

$$\arg \min_{X,U} \sum_{n=1}^{N_{hor}-1} \tau^\top \text{diag}(w_1) \tau + \varepsilon_x^\top \text{diag}(w_2) \varepsilon_x \quad (1)$$

subject to

$$\varepsilon_x = x_n - x_{des,n} \quad (2)$$

$$x_{n+1} = x_n + \dot{x}_n(x_n, u) * dt \quad (3)$$

$$x_n < \text{Lower Bound} \quad (4)$$

$$x_n > \text{Upper Bound}, \quad (5)$$

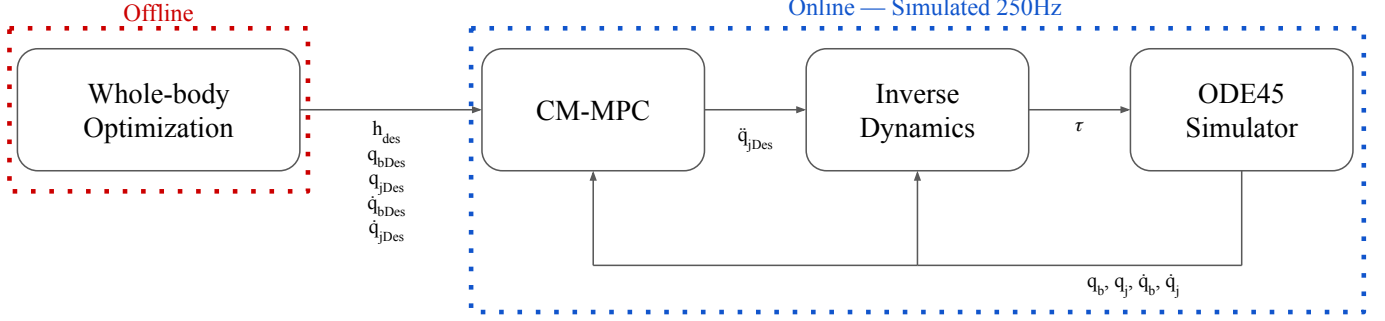


Fig. 1. Illustration of the overall system. An offline trajectory optimization feeds reference trajectories into a simulated online CM-MPC and Inverse Dynamics controller to control the robot.

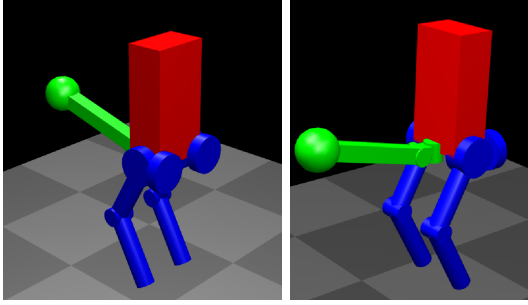


Fig. 2. Robot model with $q_j = [0 \ .25 \ 0 \ .5 \ -1 \ 0 \ .5 \ -1]^\top$

where $u_i \in \mathbb{R}^8$ is desired joint torques at the i th knot point,

$$U = [u_1 \ u_2 \ \dots \ u_{N_{hor}-1}] \quad (6)$$

$$x_n = [q_{bn}^\top \ q_{jn}^\top \ \dot{q}_{bn}^\top \ \dot{q}_{jn}^\top]^\top \quad (7)$$

$$X = [x_1 \ x_2 \ \dots \ x_{N_{hor}}]. \quad (8)$$

Dynamics in (3) are encoded using the function `FDcrb` in the package `Spatial V2` [3], which uses a composite-rigid-body algorithm, and the NLP is implemented with `CasAdi` [4] and solved using `IPOPT` [5].

The solved trajectories are used as desired state trajectories for the MPC controller in section II-D. The trajectories are interpolated from 50Hz to 250Hz using the 1-D interpolation function `griddedInterpolant`.

C. Centroidal Matrix Formulation

The centroidal momentum, $h_G \in \mathbb{R}^6$ of a robot refers to the sum of all link momenta, projected to the center of mass frame of the robot [6]. We will first define the concatenated spatial velocity of all links as $v = [v_1 \ v_2 \ \dots \ v_8]^\top$, and the spatial velocity of a specific link as $v_i \in \mathbb{R}^6$, and find the relation

$$v_i = \begin{bmatrix} \omega_i \\ \nu_i \end{bmatrix} = {}^i X_p(i) v_{p(i)} + \Phi_i \dot{q}_i \quad (9)$$

where $\omega_i \in \mathbb{R}^3$ is the angular velocity, $\nu_i \in \mathbb{R}^3$ is the linear velocity, and $\Phi_i \in \mathbb{R}^6$ is the motion subspace matrix, which selects the axis in which a joint acts on. For example, a revolute joint in the y direction has $\Phi_{Ry} = [0 \ 1 \ 0 \ 0 \ 0 \ 0]^\top$.

We can shuffle (9) to get (10), the difference of velocity between a link and its predecessor.

$$v_i - {}^i X_p(i) v_{p(i)} = \Phi_i \dot{q}_i \quad (10)$$

Then, we find the relationship between v and $\dot{q}$ to be

$$Pv = \Phi \dot{q} \quad (11)$$

$$P_{ij} = \begin{cases} \mathbf{1}^{6 \times 6} & , j = i \\ -{}^i X_p(i) v_{p(i)} & , j = p(i) \\ \mathbf{0}^{6 \times 6} & , \text{else} \end{cases} \quad (12)$$

$$\Phi = \begin{bmatrix} \Phi_1 & \mathbf{0}^6 & \dots & \mathbf{0}^6 \\ \mathbf{0}^6 & \Phi_2 & \dots & \mathbf{0}^6 \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}^6 & \mathbf{0}^6 & \dots & \Phi_8 \end{bmatrix} \quad (13)$$

where $P \in \mathbb{R}^{84 \times 84}$ is a nonstandard incidence matrix, and $\Phi \in \mathbb{R}^{84 \times 14}$ is the diagonalized joint selection matrix.

We can also find the relation $h = Iv$ between $h \in \mathbb{R}^{84}$, the system momentum matrix and $I \in \mathbb{R}^{84 \times 84}$, with

$$h = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_8 \end{bmatrix} \quad (14)$$

$$I = \begin{bmatrix} I_1 & \mathbf{0}^6 & \dots & \mathbf{0}^6 \\ \mathbf{0}^6 & I_2 & \dots & \mathbf{0}^6 \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}^6 & \mathbf{0}^6 & \dots & I_8 \end{bmatrix} \quad (15)$$

where $h_i \in \mathbb{R}^6$ is the link momentum in the world frame, and $I_i \in \mathbb{R}^{6 \times 6}$ is the link inertia. In order to find the centroidal momentum, we must transform the link momentum from the world frame into the CoM frame, and then sum it

$$h_G = \sum_{i=1}^8 {}^i\mathbf{X}_G^T h_i = \sum_{i=1}^8 {}^i\mathbf{X}_G^T I_i v_i \quad (16)$$

Using matrix manipulation to substitute the summation, we find

$$h_G = \underbrace{\mathbf{X}_G^T I P^{-1} \Phi}_{A_G(q)} \dot{q} \quad (17)$$

where $\mathbf{X}$ is the projection matrix from centroidal coordinates to all link coordinates,

$$\mathbf{X}_G = \begin{bmatrix} {}^1X_G^T \\ {}^2X_G^T \\ \vdots \\ {}^8X_G^T \end{bmatrix} \quad (18)$$

From (17), we can see that the centroidal momentum is linear in terms of joint velocity $\dot{q}$, which means we can find the centroidal momentum matrix $A_G(q) \in \mathbb{R}^{6 \times 14}$, for which $h_G = A_G(q)\dot{q}$.

D. Centroidal MPC Formulation

The centroidal MPC uses linearized centroidal dynamics as a reduced order model for the robot in the NLP. We formulated the centroidal dynamics similarly to Zhou et al. [7], with some changes to fit the robot model and centroidal momentum derivation.

Using (17), we can split q into q_b and q_j to find

$$h_G = \underbrace{[A_b(q) \ A_j(q)]}_{A_G} \begin{bmatrix} q_b \\ q_j \end{bmatrix}, \quad (19)$$

where $A_b(q) \in \mathbb{R}^{6 \times 6}$ and $A_j(q) \in \mathbb{R}^{6 \times 8}$. We can rearrange (19) into

$$\dot{q}_b = \begin{bmatrix} \underbrace{A_b^{-1}(q)}_{\widetilde{A}_b(q)} \underbrace{A_b^{-1}(q) A_j(q)}_{\widetilde{A}_j(q)} \end{bmatrix} \begin{bmatrix} h \\ \dot{q}_j \end{bmatrix}. \quad (20)$$

We can define an augmented state $x_a = [h^\top \ q_b^\top \ q_j^\top \ \dot{q}_j^\top]^\top$. Using (20) and x_a , we can formulate continuous time augmented state centroidal dynamics as:

$$\begin{bmatrix} \dot{h} \\ \dot{q}_b \\ \dot{q}_j \\ \ddot{q}_j \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \widetilde{A}_b(q) & \mathbf{0} & \mathbf{0} & \widetilde{A}_j(q) \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}}_{H_G(q)} \begin{bmatrix} h \\ q_b \\ q_j \\ \dot{q}_j \end{bmatrix} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{I} \end{bmatrix}}_{G_G} u_a, \quad (21)$$

where $u_a = \ddot{q}_{j,des}$ is the desired joint accelerations. Working under the assumption that q closely follows q_{des} , as calculated in the trajectory optimization, we can substitute the reference trajectories into (21):

$$\dot{x}_a = H_G(q_{des})x_a + G_G u_a. \quad (22)$$

We then discretize (22):

$$x_{i+1} = \widetilde{H}_{G_i} x_i + \widetilde{G}_{G_u} u_i \quad (23)$$

$$\widetilde{H}_{G_i} = H_G(q_{des,i}) * dt + \mathbf{I}^{28 \times 28} \quad (24)$$

$$\widetilde{G}_G = G_G \times dt. \quad (25)$$

We can then formulate an MPC using (23) as the dynamics constraint:

$$\arg \min_{X_a, U_a} \sum_{n=1}^{N_{hor}-1} u_n^\top \text{diag}(w_1) u_n + \varepsilon_n^\top \text{diag}(w_2) \varepsilon_n \quad (26)$$

subject to

$$\varepsilon_n = x_n - x_{des,n} \quad (27)$$

$$x_{i+1} = \widetilde{H}_{G_i} x_i + \widetilde{G}_{G_u} u_i \quad (28)$$

$$x_n < \text{Lower Bound} \quad (29)$$

$$x_n > \text{Upper Bound}. \quad (30)$$

Although (26) is linear, we still opted to solve it using Ipopt and CasAdi, similar to the trajectory optimization problem mentioned above. For each solution of the NLP, we use only u_i for the inverse dynamics controller. X_a and U_a are used to warm start the next MPC iteration.

E. Inverse Dynamics Controller and Simulation

The last stage of the controller, inverse dynamics control, is achieved using the robot dynamics

$$\tau = H(\dot{q}, q) * \ddot{q}_{des} + C(\dot{q}, q) \quad (31)$$

where $H \in \mathbb{R}^{8 \times 8}$ is the joint space inertia matrix, and $C \in \mathbb{R}^8$ is the Coriolis matrix. H and C matrices are found using the `HandC` function included in the package `Spatial V2`.

Simulation is done using a discretized model and the `ode45` [8] function supplied with MATLAB. Specifically, we use a zero-order hold on the inputs, then simulate the system forward until the end of the time step. The state is then updated to the final state of the simulation, and the simulation repeated with more inputs generated from the centroidal MPC. A pseudocode of the main control and simulation loop is provided in Algorithm 1.

III. RESULTS

IV. CONCLUSION

REFERENCES

- [1] T. Libby, A. M. Johnson, E. Chang-Siu, R. J. Full, and D. E. Koditschek, "Comparative design, scaling, and control of appendages for inertial reorientation," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1380–1398, 2016.
- [2] H. Dai, A. Valenzuela, and R. Tedrake, "Whole-body motion planning with centroidal dynamics and full kinematics," in *2014 IEEE-RAS International Conference on Humanoid Robots*. IEEE, 2014, pp. 295–302.
- [3] R. Featherstone, *Rigid body dynamics algorithms*. Springer, 2008.
- [4] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [5] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.

Algorithm 1 Main To, Simulation, and Control Process

Input $X_{des}, w_{1,TO}, w_{2,TO}, w_{1,MPC}, w_{2,MPC}$
 $dt_{TO} \leftarrow 0.02$
 $N_{hor,TO} \leftarrow 100$
 $dt_{MPC} \leftarrow 0.004$
 $N_{hor,MPC} \leftarrow 50$
function MAKEAUGMENTEDSTATE(x)
 $[q_b, q_j, \dot{q}_b, \dot{q}_j] \leftarrow \text{deal}(x)$
 $q \leftarrow [q_b; q_j]$
 $\dot{q} \leftarrow [\dot{q}_b; \dot{q}_j]$
 $h \leftarrow A_G(q)\dot{q}$ **return** $[h, q_b, q_j, \dot{q}_j]$
end function
 $x_i \leftarrow \mathbf{0}^{28}$
 $x_{i,aug} \leftarrow \text{makeAugmentedState}(x_i)$
 $[X_{ref}] \leftarrow \text{TO}(X_{des}, w_{1,TO}, w_{2,TO})$
for $i < dt_{TO} \times N_{hor,TO}/dt_{MPC}; i \leftarrow i + 1$ **do**
 $u_a \leftarrow \text{CM-MPC}(x_{i,aug}, X_{ref}, w_{1,MPC}, w_{2,MPC})$
 $\tau \leftarrow \text{InvDynamics}(u_a, x_i)$
 $[t_{sim}; x_{sim}] = \text{ODE45}(x_i, \tau)$
 $x_i \leftarrow x_{sim}(:, 0)$
 $x_{i,aug} \leftarrow \text{makeAugmentedState}(x_i)$
end for

- [6] D. E. Orin and A. Goswami, "Centroidal momentum matrix of a humanoid robot: Structure and properties," in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2008, pp. 653–659.
- [7] Z. Zhou, B. Wingo, N. Boyd, S. Hutchinson, and Y. Zhao, "Momentum-aware trajectory optimization and control for agile quadrupedal locomotion," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7755–7762, 2022.
- [8] L. F. Shampine and M. W. Reichelt, "The matlab ode suite," *SIAM journal on scientific computing*, vol. 18, no. 1, pp. 1–22, 1997.